

Overview

Part 06 — 🤖 AI Agents · 06-ai-agents/README.md



Module 06 — AI Agents

Where you are: Stop 6 of 16. **Time:** ~18 hours · **Prereq:** Module 04 — LLM [04-llm] — you need to already understand context windows, prompting and decoding before adding a loop on top of them.

A chatbot answers one turn. An **agent** is a chatbot wired into a loop that lets it act on its own outputs — call tools, read the results, and try again — without a human approving every step. This module is the theory and mechanics of **one** agent: the ReAct pattern, tool-calling mechanics, planning strategies, memory, the control loop, and the failure modes and security surface that come from letting a model take multiple unsupervised steps. Multiple agents coordinating is a different, harder problem — that's Module 07 — Agentic AI Systems [07-agentic-ai], which assumes this module. The concrete runtime that implements everything here durably in production is `../16-ai-tech-stack/tracks/langgraph/` — this module is the *why*, that track is the *how it survives production*.

Learning objectives

1. Explain precisely what turns an LLM into an "agent" — and why a chatbot with a nice UI is not one.
2. Write out the ReAct thought/action/observation prompt structure from memory, and say why interleaving beats either reasoning-only or acting-only.
3. Describe the tool-call round trip mechanically, including why it's next-token prediction, not "true" decision-making — and design a tool description that avoids mis-selection.
4. Pick a planning strategy (single-shot, Plan-and-Execute, reactive re-planning, Tree of Thoughts, Reflexion) for a given task and defend it on latency and failure-surface grounds, not vibes.
5. Name the four kinds of agent memory, map each to a real implementation, and say which one you're missing when an agent forgets something it shouldn't.
6. Compute the compounding-error math for an N-step agent at a given per-step accuracy, and use it to argue for fewer, better-scoped steps.
7. Identify the "lethal trifecta" in a proposed agent design before it ships, and know which module owns the mitigation.

Suggested path

#	Do this	File	Time
1	Agent definition, tool mechanics, ReAct, control loop, memory	notes/01-core-concepts.md	3h
2	Planning, tool design, failure modes, evals, security surface	notes/02-planning-failure-and-evals.md	3h
3	Build the ReAct loop by hand	code/react_loop_from_scratch.py	2h
4	Prove the compounding-error math	code/compounding_error_simulator.py	1h
5	Slides	slides/	1h
6	Lab	lab/EXERCISES.md	5h
7	Papers	papers/PAPERS.md	3h

The 14 terms you must own

ReAct · tool call / function calling · control loop · max-iteration guard · Plan-and-Execute · Tree of Thoughts · Reflexion / self-critique · working memory · episodic memory · semantic memory · procedural memory · compounding error · trajectory evaluation · lethal trifecta

Exit check

You can hand a colleague a one-page design note for a proposed agent: which tools it needs (with descriptions written to avoid mis-selection), which planning strategy you chose and why, how many steps the task realistically needs and what the resulting end-to-end success probability is at a stated per-step accuracy, how you'd evaluate it (per-step and trajectory, not just outcome), and whether it qualifies for the lethal trifecta — and if so, what you're doing about it before it touches production.